

Dinámica y movimiento para videojuegos y gamificación

Tema 2. Dinámica de la Partícula

Pedro Ángel Bernaola Galván

Febrero 2026

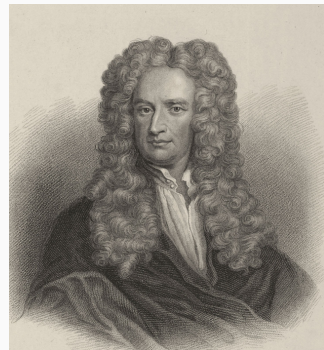
Contenido

- 1 Introducción
- 2 Las leyes de Newton
- 3 Getting started con pymunk y pybullet
- 4 Fuerzas gravitatorias
- 5 Fuerza elástica
- 6 Rozamiento
- 7 Trabajo y energía

1 Introducción

Introducción

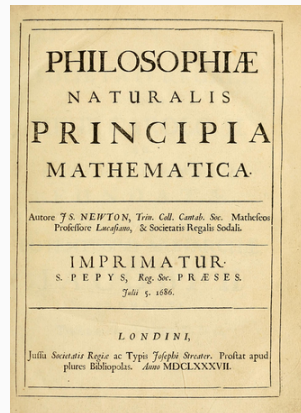
- El capítulo analiza las leyes que relacionan la fuerza, la masa y el movimiento.
- En primer lugar veremos las **Leyes de Newton**.
- Estas leyes son la base de la mecánica clásica y fundamentales para cualquier motor de física.
- **Concepto clave:** La fuerza es una cantidad vectorial (tiene magnitud y dirección).
- Veremos fuerzas que aparecen con frecuencia en las simulaciones: **gravitatoria, de rozamiento y fuerzas elásticas**.



2 Las leyes de Newton

Leyes de Newton

1. **Primera Ley:** Ley de la Inercia.
 2. **Segunda Ley:** Ley Fundamental de la Dinámica
($F = ma$).
 3. **Tercera Ley:** Ley de Acción y Reacción.
- **Publicación:** 5 de julio de 1687.
 - **Obra:** *Philosophiæ Naturalis Principia Mathematica*.
 - **Impacto:** Establecieron las bases de la mecánica clásica que usamos hoy en simulaciones.



Primera Ley de Newton: Inercia

- Un objeto en reposo permanece en reposo y un objeto en movimiento permanece en movimiento a velocidad constante a menos que actúe sobre él una fuerza externa neta.
- La **masa** es la medida de la inercia de un objeto (lo fácil o difícil que es cambiar su velocidad).



$$\text{Si } \sum \vec{F} = 0 \implies \vec{a} = 0 \implies \vec{v} = cte$$



Primera Ley de Newton: Inercia

- Debido a la inercia, en el espacio es tan difícil acelerar un objeto como detenerlo: ambos requieren una fuerza externa.
- Las naves regresan a velocidades orbitales extremas (aprox. **28,000 km/h**).
- No disponen de combustible suficiente para frenar usando motores; requerirían un cohete tan grande como el de salida.
- La solución es utilizar la **fricción atmosférica** como "freno natural", transformando la energía cinética en calor intenso.



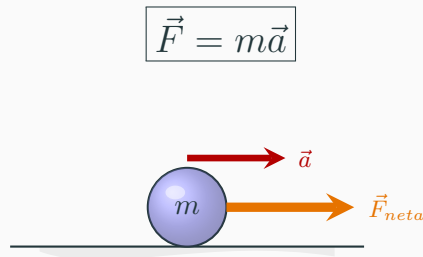
La atmósfera como freno.

Segunda Ley de Newton

- La **Segunda Ley** define la fuerza como la causa del cambio de movimiento.
- El efecto de una fuerza sobre la velocidad es inversamente proporcional a la masa (inercia).

$$\vec{a} = \frac{\sum \vec{F}_{neta}}{m}$$

- Se considera la definición operacional de la **Fuerza**.
- Sus unidades $[F] = \text{kg} \frac{\text{m}}{\text{s}^2} \equiv \text{N}$ (newton)



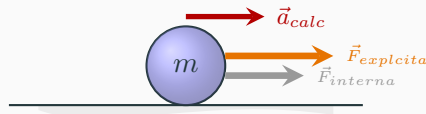
Segunda Ley: La Realidad del Motor Físico

- **Caja Negra:** Algunos motores (Pymunk/PyBullet) ocultan las *fuerzas internas* (fricción, normal, impulsos de colisión).
- No solemos tener acceso a la *fuerza total* (el motor oculta fricciones y colisiones internas).
- El programador solo inyecta **fuerzas explícitas**.
- La aceleración no es un dato, es una consecuencia numérica:

Cálculo de la aceleración:

$$\vec{a}_{calc} \approx \frac{\vec{v}_n - \vec{v}_{n-1}}{\Delta t}$$

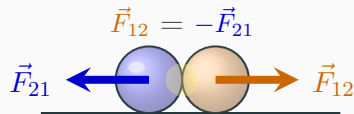
Derivada numérica utilizada en la simulación.



Tercera Ley de Newton: Acción y Reacción

- Siempre que un objeto ejerce una fuerza sobre otro, el segundo ejerce una fuerza de igual magnitud y dirección opuesta.
- **Propulsión a chorro:** El cohete acelera hacia adelante al empujar los gases de escape hacia atrás.
- **Importante:** No se empuja “contra el suelo”; la interacción es entre el motor y el gas. Por eso los cohetes funcionan en el vacío del espacio.
- *"Si el cohete empuja al gas, el gas empuja al cohete".*

$$\vec{F}_{\text{cohete}} = -\vec{F}_{\text{gases}}$$



Acción y reacción en el despegue.

3 Getting started con pymunk y pybullet

Tutorial Pymunk (1): Configuración de la ventana

Configuración del entorno:

- `import`: Cargamos `pymunk` para el motor de física y `pygame` para la renderización gráfica.
- `pygame.init()`: Prepara el hardware y los módulos (sonido, video, eventos) para trabajar.
- `set_mode`: Define el lienzo de dibujo. En este caso, un cuadrado de 600×600 píxeles.

Código Python

```
1 import pymunk
2 import pygame
3 #1. Configuración de la ventana
4 pygame.init()
5 pantalla = pygame.display.set_mode((600, 600))
6 pygame.display.set_caption("Hello Munk")
7 reloj = pygame.time.Clock()
8
```

- `set_caption`: Establece el nombre de la aplicación en la barra de tareas.
- `Clock()`: Elemento crítico para sincronizar los pasos físicos con los frames visuales.

Tutorial Pymunk (2): Mundo, Suelo y Cuerpo Rígido

El motor físico:

- **Space:** Es el “Mundo”.
- **Suelo (Segment):** Un cuerpo estático (static_body). No es obligatorio.
- **Caja (Body):**
 - **Masa y Momento:** Necesarios para que el motor sepa cómo se desplaza y cómo rota.
 - **Posición:** Se sitúa en (300, 50) (parte superior central).
- **add:** Es vital añadir tanto el Body (física) como la Shape (forma) al espacio.

Código: Mundo y Objetos

```

1  # 2. Configuración del mundo físico
2  espacio = pymunk.Space()
3  # --- CREAR EL SUELO ---
4  suelo = pymunk.Segment(espacio.static_body,
5  (0, 550), (600, 550), 5)
6  suelo.elasticity = 0.5
7  espacio.add(suelo)
8  # --- CREAR LA CAJA ---
9  masa = 1
10 lado = 50
11 momento = pymunk.moment_for_box(masa, (lado, lado))
12 cuerpo_caja = pymunk.Body(masa, momento)
13 cuerpo_caja.position = (300, 50)
14 forma_caja = pymunk.Poly.create_box(cuerpo_caja, (
15     lado, lado))
16 forma_caja.elasticity = 0.8
17 espacio.add(cuerpo_caja, forma_caja)

```

¡OJO!: distingue entre el cuerpo (body) y la forma (shape)

Tutorial Pymunk (3): Bucle y Renderizado

Actualización y Dibujo:

- `while running`: Mantiene el programa activo hasta cerrar la ventana.
- `espacio.step()`: Avanza la simulación física. Se recomienda un paso fijo (1/60) para mayor estabilidad.
- **Renderizado:**
 - `fill`: Limpia la pantalla cada frame.
 - `pygame.draw.rect`: Dibuja la caja usando la posición del cuerpo de Pymunk.
- `clock.tick(60)`: Sincroniza el bucle a 60 cuadros por segundo.

```
1 # 3. Bucle principal
2 running = True
3 while running:
4     for event in pygame.event.get():
5         if event.type == pygame.QUIT:
6             running = False
7     pantalla.fill((255, 255, 255))
8     # Avanzar fisica
9     espacio.step(1/60.0)
10    # Dibujar caja (posicion actual)
11    pos = cuerpo_caja.position
12    pygame.draw.rect(pantalla, (255, 0, 0),
13                     (pos.x-25, pos.y-25, 50, 50))
14    # Dibujar suelo
15    pygame.draw.line(pantalla, (0, 0, 0),
16                     (0, 550), (600, 550), 5)
17    pygame.display.flip()
18    reloj.tick(60)
19 pygame.quit()
20
```

Tutorial Pymunk (4): Métodos de Dibujo

1. Dibujo Manual (Pygame): Existen funciones específicas según la forma física:

- `draw.circle()`: Para círculos.
- `draw.line()`: Para segmentos/suelos.
- `draw.polygon()`: Para cajas o formas irregulares.

Nota: Se calculan rotaciones y centros manualmente.

2. Dibujo Automático (Debug): Pymunk ofrece `DrawOptions`, que sincroniza automáticamente la posición y la **rotación** de todos los objetos en el espacio.

Aquí `pantalla` es el objeto tipo `surface` que creamos al principio al llamar a

`pygame.display.set_mode((600, 600))`

Uso de Debug Draw

```

1  import pymunk.pygame_util
2
3  # 1. Configurar opciones (fuera del
   bucle)
4  opciones = pymunk.pygame_util.
   DrawOptions(pantalla)
5
6  # 2. En el bucle principal
7  while running:
8      # ... eventos y step ...
9
10     pantalla.fill((255, 255, 255))
11
12     # Dibuja todo el espacio
   automaticamente
13     espacio.debug_draw(opciones)
14
15     pygame.display.flip()
16

```


Tutorial Pybullet (1): Mundo y Suelo 3D

Entorno 3D:

- `p.GUI`: Inicia la simulación con una ventana gráfica 3D interactiva.
- `getDataPath()`: Configura la ruta para encontrar modelos estándar (como el suelo).
- `loadURDF`: Carga objetos desde archivos. El *plane.urdf* es el suelo infinito.
- `restitution`: Es el término técnico para el “rebote”. El valor `-1` indica que se aplica a la base del objeto.

Código: Inicialización Pybullet

```
1  import pybullet as p
2  import pybullet_data
3
4  # 1. Configuración inicial
5  p.connect(p.GUI)
6  p.setAdditionalSearchPath(
7      pybullet_data.getDataPath())
8
9  # 2. Cargar suelo y configurar rebote
10 suelo_id = p.loadURDF("plane.urdf")
11 # restitution = elasticidad
12 p.changeDynamics(suelo_id, -1,
13     restitution=0.7)
```

Tutorial Pybullet (2): Objeto y Cámara

Cuerpos y Visualización:

- `loadURDF`: Cargamos el cubo pequeño. El segundo argumento `[0, 0, 1.0]` establece su posición inicial (X, Y, Z) . Al estar en $Z = 1$, caerá un metro.
- `restitution`: Al igual que con el suelo, asignamos un rebote del 70%. Si ambos tienen restitución, el efecto se combina: **es el producto de las dos.**

Código: Cubo y Cámara

```

1  # 3. Cargar cubo y configurar su rebote
2  # Posicion Z=1.0 (1 metro de altura)
3  cubo_id = p.loadURDF("cube_small.urdf", [0, 0, 1.0])
4  p.changeDynamics(cubo_id, -1, restitution=0.7)
5
6  # 4. Ajustar camara
7  p.resetDebugVisualizerCamera(
8      cameraDistance=1.5,
9      cameraYaw=45,
10     cameraPitch=-30,
11     cameraTargetPosition=[0, 0, 0])
12

```

- **Cámara:**
 - `cameraDistance`
 - `Yaw/Pitch`
 - `TargetPosition`

Tutorial Pybullet (3): Bucle de Simulación

Ejecución de la Física:

- `stepSimulation`: Ordena al motor calcular el siguiente estado (posiciones, colisiones, velocidades).
- `time.sleep`: Pybullet corre a 240Hz por defecto. Pausamos 1/240 segundos para que la simulación coincida con el tiempo real.
- **Cálculo del Rebote:**
 - La restitución final se calcula como el producto de ambas:

$$e_{total} = e_{suelo} \times e_{cubo}.$$
 - En este ejemplo: $0,7 \times 0,7 = 0,49$.

Código: El Bucle Infinito

```

1      import time
2      # 5. Bucle de simulacion
3      print("Simulacion iniciada...")
4      try:
5          while True:
6              # Avanza el motor fisico
7              p.stepSimulation()
8              # Sincroniza con tiempo real (240Hz)
9              time.sleep(1./240.)
10     except KeyboardInterrupt:
11         # Cierra la conexion al salir
12         p.disconnect()
13         print("\nConexion cerrada.")
14

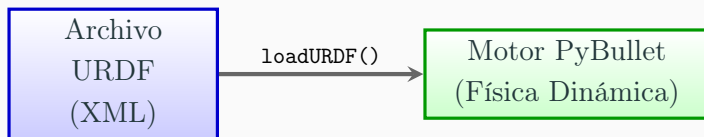
```

- **Control de salida:** El bloque `try/except` asegura que la conexión se cierre limpiamente al interrumpir.

Tutorial Pybullet (4): Archivos .urdf

Al cargar un archivo .urdf en PyBullet mediante `loadURDF()`, el motor no solo importa la geometría visual, sino que extrae propiedades físicas críticas definidas en el XML:

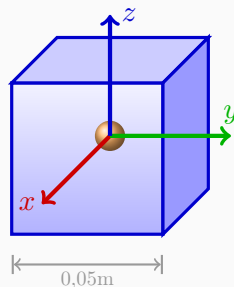
- **Masa e Inercia:** Definidas en la etiqueta `<inertial>`, fundamentales para la simulación de la dinámica.
- **Fricción y Restitución:** Parámetros de contacto especificados en `<contact>`.
- **Límites de Articulación:** Esfuerzo máximo, velocidad y límites de posición en los `<joint>`.



Tutorial Pybullet (5): Propiedades de `cube_small.urdf`

El archivo `cube_small.urdf` es un estándar en PyBullet para tareas de manipulación. Sus propiedades físicas están optimizadas para la estabilidad en el contacto:

- **Dimensiones:** Cubo de $5 \times 5 \times 5$ cm (0,05 m).
- **Masa e Inercia:** 0,05 kg; $I_{i,i} \approx 2,08 \times 10^{-5}$ kg/m².
- **Fricción:** Coeficientes elevados para evitar deslizamientos en el agarre.
- **Extracción Dinámica:** Usamos `getDynamicsInfo()` para obtener masa, fricción e inercia del link.
- **Datos Visuales:** `getVisualShapeData()` permite recuperar las dimensiones y escala de la malla.



Tutorial Pybullet (6): Fichero cube_small.urdf

Si los archivos no han sido creados o descargados manualmente, PyBullet utiliza sus propios archivos estándar ubicados en el paquete de datos:

Ruta del sistema:

```
<entorno>/lib/python3.10/
site-packages/pybullet_data
```

- Contiene geometrías (.obj, .stl).
- Incluye robots (Panda, Kuka).
- **Carga:** Se accede mediante la librería `pybullet_data`.

```

1  <?xml version="0.0" ?>
2  <robot name="cube.urdf">
3  <link name="baseLink">
4  <contact>
5  <lateral_friction value="1.0"/>
6  </contact>
7  <inertial>
8  <mass value=".1"/>
9  <inertia ixx="1" ixy="0" ixz="0"
10 iyy="1" iyz="0" izz="1"/>
11 </inertial>
12 <visual>
13 <geometry>
14 <mesh filename="cube.obj"
15 scale=".05 .05 .05"/>
16 </geometry>
17 </visual>
18 <collision>
19 <geometry>
20 <box size=".05 .05 .05"/>
21 </geometry>
22 </collision>
23 </link>
24 </robot>
25
```

Renderizado: Pymunk vs. Pybullet

Gestión Visual:

- **Pymunk:** Es un motor “ciego”. Solo calcula números. Necesitas una librería externa (**Pygame**) y programar cada dibujo manualmente.
- **Pybullet (GUI):** Es un entorno integral. Al conectar con `p.GUI`, el motor de renderizado OpenGL se encarga de todo.

El Modo `p.DIRECT`:

- Ejecuta la física **sin ventana gráfica**.
- **Uso principal:** Entrenamiento de Redes Neuronales y Reinforcement Learning.
- **Ventaja:** Es mucho más rápido al no gastar recursos en procesar imágenes.

Flujo de Trabajo

```
1  # EN PYMUNK + PYGAME
2  # Tienes que "pintar" cada frame
3  pygame.draw.rect(pantalla, rojo,
4                   pos_caja)
5  # EN PYBULLET (GUI)
6  # El renderizado es automatico
7  p.stepSimulation()
8  # EN PYBULLET (DIRECT)
9  # Solo calculos, ideal para
10 servidores
11 p.connect(p.DIRECT)
12 for _ in range(10000):
13     p.stepSimulation()
```

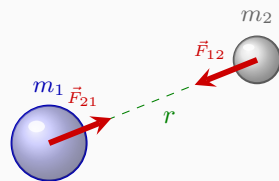
4 Fuerzas gravitatorias

Ley de Gravitación Universal

- Existe una fuerza de atracción entre cualquier par de objetos con masa.
- Esta fuerza es:
 - Proporcional al producto de sus masas.
 - Inversamente proporcional al cuadrado de la distancia (r).

$$F_G = G \frac{m_1 m_2}{r^2}$$

- $G = 6,674 \times 10^{-11} \text{ N} \cdot \text{m}^2/\text{kg}^2$
- **Aplicación:** Crucial para calcular trayectorias de cohetes y órbitas satelitales (como en la misión Artemis).



Trayectorias de la misión Artemis.

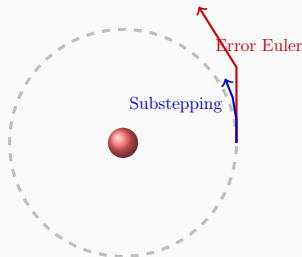
gravitacion03.py

Estabilidad Numérica: Substepping

Las fuerzas centrales son extremadamente sensibles al paso de tiempo (dt).

El *substepping* mejora la fidelidad (véase `gravitacion01.py` y `gravitacion02.py`):

- **Error de Discretización:** En pasos grandes, el objeto “escapa” de la curva real al moverse en líneas rectas demasiado largas.
- **División del Tiempo:** Dividir el `step` de un frame en N sub-pasos reduce el error de energía acumulado.
- **Implementación:**
 - Se itera N veces el cálculo de fuerza.
 - Se ejecuta `space.step(dt/N)`.
- **Resultado:** Órbitas estables sin necesidad de integradores complejos.

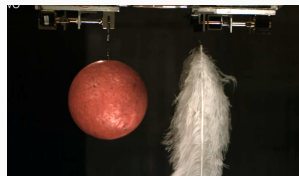
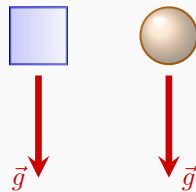


Gravedad en la Tierra (g)

- La aceleración de la gravedad es independiente de la masa del objeto que cae.
- En la superficie terrestre, simplificamos la Ley Universal como:

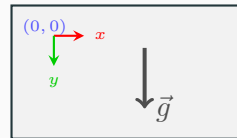
$$P = F_g = mg$$

- Donde $g \approx 9,81 \text{ m/s}^2$.
- **Concepto clave:** En ausencia de aire (vacío), una bola y una pluma caen exactamente a la misma velocidad.
- En otros cuerpos celestes el valor cambia (p.ej. Luna: $1,62 \text{ m/s}^2$).



Implementación: Pymunk (2D)

- **Espacio de trabajo:** Se define a nivel global en el objeto `Space`.
- **Unidades:** ¡Cuidado! Pymunk no usa metros, usa **píxeles**.
- **Escala:** Para una gravedad natural, se usan valores entre 600 y 1000 px/s^2 .
- Las velocidades resultantes se medirán en px/s .



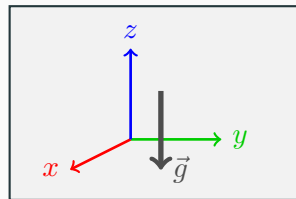
Y aumenta hacia abajo

Código Python

```
1 import pymunk
2 space = pymunk.Space()
3 # g en px/s^2 (vector x, y)
4 space.gravity = (0, 900)
5
```

Implementación: PyBullet (3D)

- **Simulación 3D:** La gravedad requiere un vector de tres dimensiones (x, y, z).
- **Eje vertical:** Por defecto, la gravedad actúa sobre el eje **Z**.
- **Unidades:** PyBullet está orientado a la robótica, por lo que usa unidades del SI (**metros/s²**).
- Objetos con `mass=0` ignoran la gravedad (son estáticos).
- Convenio de colores de los ejes (X, Y, Z) $\rightarrow$ (R, G, B)



Código Python

```
1 import pybullet as p
2 p.connect(p.GUI)
3 # g en m/s^2 (vector x, y, z)
4 p.setGravity(0, 0, -9.81)
5
```

Comparativa: Pymunk vs. PyBullet

Característica	Pymunk (2D)	PyBullet (3D)
Unidades	Píxeles	Metros
Gravedad típica	(0, 900)	(0, 0, -9,81)
Visualización	Externa (Pygame)	Nativa (p.GUI)
Uso principal	Juegos 2D, teoría	Robótica, juegos 3D

Consejo para desarrolladores

Si vuestro proyecto es en 2D, **Pymunk** es mucho más ligero y fácil de integrar. Para simulaciones físicas realistas o robótica, **PyBullet** es el estándar de la industria.

Ejemplo. Tiro Parabólico

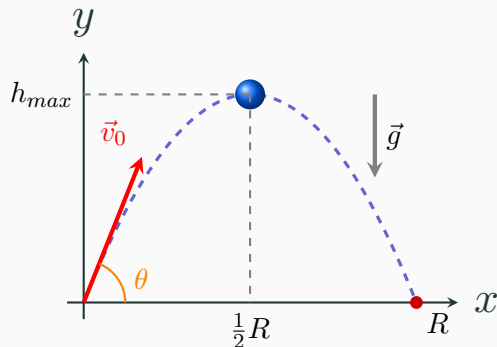
El movimiento de un proyectil bajo la aceleración constante de la gravedad g se descompone en dos ejes independientes:

Eje Horizontal (MRU):

- $v_x = v_0 \cos \theta$
- $x = x_0 + v_x t$

Eje Vertical (MRUA):

- $v_y = \underbrace{v_0 \sin \theta}_{v_{0y}} - gt$
- $y = y_0 + v_{0y}t - \frac{1}{2}gt^2$



Ejemplos: `disparo02.py` (Pymunk) `disparo3D_02.py` (PyBullet)

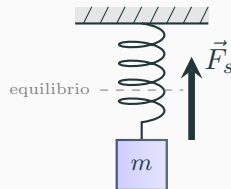
5 Fuerza elástica

Fuerza Elástica: Ley de Hooke

- Modela la fuerza ejercida por muelles o materiales elásticos.
- La fuerza (*aproximadamente*) es proporcional al desplazamiento (x) desde la posición de equilibrio:

$$F_s = -kx$$

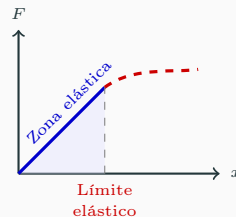
- Donde:
 - k : Constante elástica (rigidez) $[k] = \text{N/m}$.
 - El signo negativo indica que es una **fuerza restauradora**.
- **Aplicación:** Fundamental para simular suspensiones de vehículos, cuerdas elásticas y algunas colisiones.
- Lo veremos en detalle luego cuando veamos el rozamiento.



Más allá de la Ley de Hooke

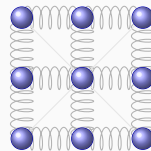
- **Límite Elástico:**

- La ley de Hooke es solo una aproximación para desplazamientos pequeños.
- Si estiramos demasiado el material (x es muy grande), la ley deja de ser lineal.
- El objeto puede sufrir deformaciones permanentes o romperse.



- **Modelado de "Soft Bodies":**

- Uniendo varias masas con muelles (k) podemos simular:
 - Ropa, banderas y pelo dinámico.
 - Gelatinas o cuerpos blandos.



Malla de Muelles (Física de Telas)

Dinámica del Péndulo Simple

La dinámica del péndulo está gobernada por la componente tangencial del peso:

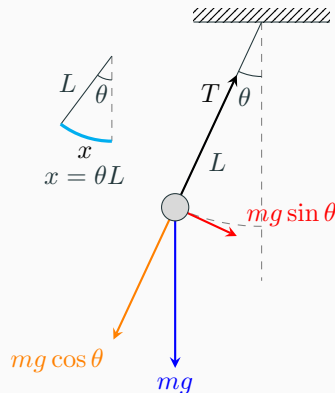
- **Fuerza recuperadora:**

$$F = -mg \sin \theta$$

Apunta siempre hacia el equilibrio.

- **Variable de arco (x):** La distancia sobre la trayectoria es $x = \theta L$.
- **Aproximación lineal:** Para ángulos pequeños ($\sin \theta \simeq \theta$):

$$F \simeq -mg\theta = -mg\frac{x}{L}$$

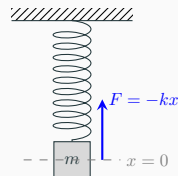


Analogía: El Péndulo vs. El Muelle

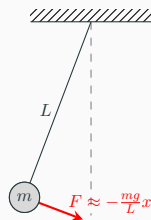
Se comporta como un muelle en el que la fuerza “elástica” es la gravedad (componente tangencial de la fuerza)

Propiedad	Muelle	Péndulo
Fuerza Recuperadora	$F = -kx$	$F \simeq -\frac{mg}{L}x$
Frecuencia Angular (ω)	$\sqrt{\frac{k}{m}}$	$\sqrt{\frac{g}{L}}$
Variable de estado	Elongación (x)	Arco ($x = L\theta$)

- Ambos sistemas son equivalentes cuando la fuerza recuperadora es linealmente proporcional al desplazamiento. Esto ocurre en el péndulo para ángulos pequeños ($\sin \theta \simeq \theta$).



Muelle (OAS)



Péndulo ($\theta \ll 1$)

6 Rozamiento

Rozamiento. Conceptos fundamentales

El **rozamiento** es cualquier fuerza que se opone al movimiento y transforma la energía mecánica en calor

- **Fricción (Sólida/Coulomb):**

- Se origina en el contacto entre superficies rugosas.
- Magnitud: $F_r \leq \mu N$ ($N \rightarrow$ fuerza normal a la superficie de contacto).
- Es independiente de la velocidad.

- **Damping (Viscoso/Amortiguamiento):**

- Se origina por la resistencia de un fluido o histéresis interna.
- Magnitud: Depende de la velocidad: $F_v = -f(v)$. Siempre opuesta a $\vec{v}$
- Se suele tomar $F_v = -bv$, aunque puede aparecer $F_v \propto v^\alpha$ con $\alpha > 1$

Nomenclatura

Usamos **Fricción** para el contacto suelo-objeto y **Damping** (rozamiento viscoso en castellano) para la resistencia del aire o el comportamiento interno de muelles y cuerdas.

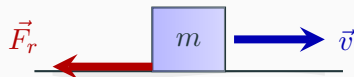
Fricción

- La fricción es la fuerza que se opone al movimiento relativo entre materiales sólidos.
- Depende de la naturaleza de las superficies (coeficiente μ) y de la fuerza normal (N)
- **Fricción Estática** (μ_e): Resistencia para iniciar el movimiento.

$$F_r \leq \mu_e N$$

- **Fricción Dinámica** (μ_d): Resistencia durante el deslizamiento ($\mu_d < \mu_e$).

$$F_r = \mu_d N$$



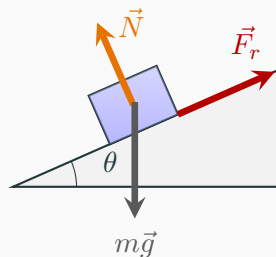
Ejemplos

`friccion01.py`

`friccion02.py`

Fricción en Planos Inclinados

- En una rampa, la fuerza normal se reduce: $N = mg \cos(\theta)$.
- Por lo tanto, la fuerza de fricción también disminuye a medida que aumenta la pendiente.
- **En videojuegos:** Fundamental para decidir si un objeto se queda estático o empieza a resbalar por una pendiente.



`friccion03.py`

Damping o rozamiento viscoso: el modelo lineal

En medios fluidos o sistemas mecánicos lubricados, la resistencia al movimiento se modela mediante fuerzas disipativas que dependen de la velocidad.

- **Definición:** La fuerza de rozamiento viscoso es directamente proporcional a la velocidad del objeto $\vec{F}_v = -b \cdot \vec{v}$

Donde b es el coeficiente de amortiguamiento o **damping**. $[b] = \text{N}\cdot\text{s}/\text{m} = \text{kg}/\text{s}$

- **Sentido:** El signo negativo indica que la fuerza siempre se opone al vector velocidad, extrayendo energía del sistema.

Analogía Física

Este modelo es equivalente al de un amortiguador hidráulico o al de una esfera pequeña moviéndose lentamente en un fluido denso (Régimen de Stokes).

Implementación en Pymunk: `space.damping`

Pymunk simplifica el cálculo de la fricción viscosa a través de un parámetro global en el objeto `Space`.

- **Mecanismo:** En lugar de calcular una fuerza en Newtons, Pymunk escala la velocidad en cada paso de tiempo (*step*).
- **Algoritmo de actualización:** $v_{\text{new}} = v_{\text{old}} * \text{damping} * dt$
- **Valores del parámetro:**
 - `damping = 1.0`: No hay pérdida de energía (vacío perfecto).
 - `damping = 0.9`: El cuerpo pierde un 10 % de su velocidad por unidad de tiempo.
- **Ventaja:** Muy eficiente, no hay que calcular fuerzas para cada objeto.
- **Inconvenientes:**
 - Es igual para todos, no se puede tener en cuenta datos reales de los cuerpos (e.g. aerodinámica, textura, forma)
 - No es directo relacionar el valor de `damping` con la constante de amortiguamiento viscoso (b)

Damping en Motores 3D: PyBullet

Al igual que Pymunk, PyBullet utiliza el modelo de **fricción viscosa lineal** para estabilizar la simulación y simular medios fluidos.

- **Desacoplamiento:** A diferencia de los motores 2D simples, permite separar el `linearDamping` del `angularDamping`.
- **Ecuación de integración:** PyBullet integra estas fuerzas de amortiguamiento dentro del *solver* de restricciones:

$$v_{t+1} = v_t - \frac{b}{m} \cdot v_t \cdot dt$$

- **Granularidad:** Se define por cada objeto (*body*) individualmente, no de manera global para todo el espacio.
- Además de simular aire, el damping en PyBullet es crítico para evitar que los robots u objetos “vibren” o ganen energía infinita debido a errores numéricos del integrador.

Muelle Amortiguado

Ecuación Diferencial

El movimiento de una masa m sujeta a un muelle con amortiguamiento lineal sigue:

$$m \frac{d^2x}{dt^2} + b \frac{dx}{dt} + kx = 0$$

Constantes Físicas:

- k : Constante elástica o rigidez.
- b : Coeficiente de amortiguamiento.
- En simulaciones, si $b^2 \simeq 4mk$, el sistema alcanza el **amortiguamiento crítico**.
- **Tiempo característico**: $\tau = \frac{1}{\gamma} = \frac{2m}{b}$

Unidades (SI vs Pymunk)

Parámetro	S.I.	Pymunk
Masa (m)	kg	unidad masa
Distancia (x)	m	píxeles
Rigidez (k)	N/m	$\frac{\text{masa}}{\text{tiempo}^2}$
Amort. (b)	N·s/m = kg/s	$\frac{\text{masa}}{\text{tiempo}}$

Nota: Pymunk es adimensional por defecto. Si usas píxeles como metros y segundos reales, las constantes escalan proporcionalmente a la masa definida.

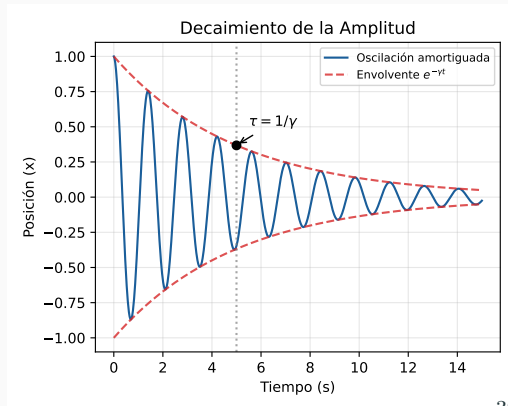
Tiempo de característico (τ)

El tiempo característico o tiempo de relajación, τ , representa la escala temporal en la que la energía del sistema se disipa de forma significativa.

Interpretación Física

$$\tau = \frac{1}{\gamma} = \frac{2m}{b}$$

- Es el tiempo necesario para que la amplitud caiga a $1/e$ ($\simeq 37\%$) de su valor inicial.
- Cuanto mayor es el amortiguamiento b , menor es τ y más rápido se detiene el sistema.
- Define la "memoria" del sistema respecto a su estado inicial.

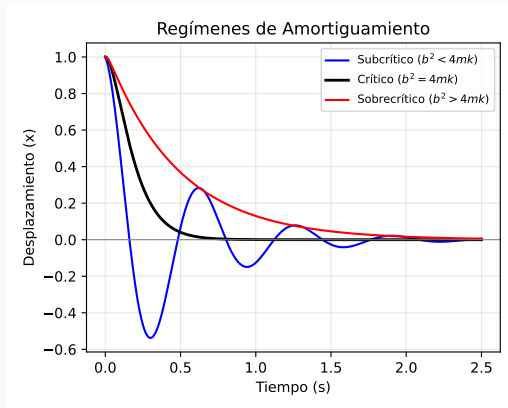


Regímenes de Amortiguamiento

La respuesta del sistema depende de la relación entre la disipación (b) y la restauración (k).

Clasificación Física

- **Subcrítico** ($b^2 < 4mk$): El sistema oscila con amplitud decreciente.
- **Crítico** ($b^2 = 4mk$): El sistema regresa al equilibrio lo más rápido posible sin oscilar.
- **Supercrítico** ($b^2 > 4mk$): Fricción excesiva. El regreso al equilibrio es lento y "viscoso".



Muelles Amortiguados: DampedSpring en Pymunk

Parámetros:

- **a y b**: Cuerpos unidos a los extremos del muelle (estáticos o dinámicos)
- **anchor_a y anchor_b**: Puntos de anclaje
- **rest_length**: Longitud del muelle en reposo.
- **stiffness**: La k del muelle (recordar las unidades “peculiares” de Pymunk)
- **damping**: Coeficiente de amortiguamiento b , el que aparece en la ecuación diferencial, no un porcentaje como en el damping espacial

muelle01.py

muelle_doble01.py

Código Python

```

1  # Creamos el muelle amortiguado
2  muelle = pymunk.DampedSpring(
3      a=cuerpo_a,
4      b=cuerpo_b,
5      anchor_a=punto_anclaje_a,
6      anchor_b=punto_anclaje_b,
7      rest_length=longitud_reposo,
8      stiffness=rigidez, # la k del
9                          muelle
10     damping=amortiguamiento # la b
11 )
12 # IMPORTANTE incluirlo en el
13 espacio
    espacio.add(muelle)

```

7 Trabajo y energía

El Concepto de Trabajo (W)

El trabajo es el resultado de la aplicación de una fuerza a lo largo de una distancia

- El trabajo se define formalmente como el **producto escalar** del vector fuerza ($\vec{F}$) por el vector desplazamiento ($\vec{d}$):

$$W = \vec{F} \cdot \vec{d} = Fd \cos \theta$$

- Donde θ es el ángulo entre la fuerza y la dirección del movimiento.
- **Interpretación:** Solo la componente de la fuerza que actúa en la dirección del movimiento realiza trabajo.
- **Unidades:** Se mide en **julios (J)**, donde $1 \text{ J} = 1 \text{ N} \cdot \text{m}$.

Energía: Cinética y Potencial

La energía es la capacidad de un sistema para realizar trabajo.

Energía Cinética (E_c)

Energía debida al movimiento. Fundamental para calcular impactos y colisiones en el motor físico.

$$E_c = \frac{1}{2}mv^2$$

Energía Potencial Gravitatoria (E_p)

Depende de la configuración del sistema bajo la fuerza de gravedad.

- **Aproximación Local:** Si **no** estamos en un problema orbital o de distancias astronómicas, usamos:

$$E_p = mgh$$

- Donde $g \approx 9,81 \text{ m/s}^2$ es la aceleración constante cerca de la superficie terrestre.

Energía Potencial Elástica (E_k)

Energía almacenada al comprimir o extender un muelle: $E_k = \frac{1}{2}k\Delta x^2$

Potencia (P)

La potencia es la proporción a la que se transfiere energía o se realiza un trabajo en el tiempo.

$$P = \frac{W}{\Delta t}$$

- Se mide en watios (W), $1 \text{ W} = 1 \text{ J/s}$
- **Visión para Vehículos (Dinámica):** Para aplicaciones como el movimiento de coches o naves, es muy útil expresarla como el **producto escalar de la fuerza por la velocidad**:

$$P = \vec{F} \cdot \vec{v}$$

- **Importancia en Gameplay:**
 - Permite calcular la potencia necesaria para mover un objeto a una velocidad dada conociendo la fuerza que se opone (e.g. rozamiento con el aire)
 - Fundamental para simular el par motor y los límites de aceleración de vehículos.